

Client-Server Sessions by Nondeterministic Recursion

ANONYMOUS AUTHOR(S)

Session types provide a robust and expressive foundation for type-safe message-passing concurrency, and Phil Wadler’s process language CP unifies session types with classical linear logic in a way that guarantees deadlock freedom. However, CP is deterministic, and the order of communication is not influenced by which participant is ready but only by the values transmitted. This is insufficient for many real-world applications. For example, chat rooms and download managers need to manage a dynamically sized pool of client processes that handle users and connections, serving them according to readiness. Multiple approaches have been proposed in the literature, but so far, the problem has resisted a unified, expressive solution based on session-typed communication between clients and servers.

In this paper, we add typed interactions between servers and pools of clients to CP by using fixed-point types for client-server sessions. Dynamically-sized client populations have least fixed-point types and servers have greatest fixed-point types. We introduce a recursive control structure, poll/submit, for servers to traverse such populations of clients based on when they are ready. Once a client is selected by the server, it interacts with the server using the already-rich protocol language of classical linear logic. We demonstrate the expressivity of our system by implementing three constructs inspired by prior work: coexponentials (permuted sessions), classical effects (request-response), and shared linear state (mutexes). We prove and mechanize in Lean that our system enjoys preservation and progress, and thereby is type-safe and deadlock-free.

1 Introduction

Wadler’s *Propositions as Sessions* showed that classical linear logic provides a foundation for session-typed concurrency and defined CP (classical processes) as a process language that is Curry–Howard isomorphic to proofs in classical linear logic [8]. In CP, propositions describe communication protocols, proofs become concurrent processes, and cut elimination becomes communication. Well-typed programs therefore guarantee protocol fidelity, linear use of channels, and deadlock freedom for closed systems.

What CP does not provide is a way to let the next communication depend on which pending interaction becomes ready. For example, a chat room may need to react to whichever participant messages first, while a download manager fetching from several mirrors in parallel may need to update aggregate progress whenever any worker reports it. In such examples, the next interaction is determined by readiness and the number of clients may be determined dynamically at runtime (e.g., when a new user joins the chat or more mirrors are discovered). However, in CP, once the values of the inputs to a system are fixed, its observable communication behavior is fixed as well, and thus its behavior cannot depend on readiness.

Several extensions of linear-logic process calculi have attempted to address this gap [4–7], but they do so with limitations. *Towards Races in Linear Logic* [4] supports only a fixed number of clients per server, so the pool cannot grow. *Client-Server Sessions in Linear Logic* [6] does not support repeated interactions with the server by a client. Qian’s dissertation [5] removes that restriction and is substantially more expressive, yet still cannot implement mutexes. *Safe Session-Based Concurrency with Shared Linear State* [7] is the most expressive among these, but is not in the form of two-sided communication channels. Instead, clients share linear state through a mutex.

In this paper, we propose a system for such readiness-based communication between a dynamic pool of clients and a single server that is based on two-sided communication channels. This system can implement the features of these other systems other than those that are not compatible with a dynamically sized pool. Specifically, using dynamically sized pools prevents us from implementing features that depend on statically knowing the number of clients, such as returning a fixed-size tuple that combines the final results of all clients.

To this end, we repurpose fixed-point types [1] by interpreting the least fixed-point as a population of clients, and the greatest fixed-point as a channel to the server. We introduce a new control structure for fixed-point types called poll/submit. A server can poll a pool of clients, interact with the one that is ready, and then submit zero or more residual clients back into that pool. The construction fits naturally into session typing because the client-server protocols are described inside the recursive fixed-point types by the already-rich protocol language of classical linear logic. In the *(AnonLang)*¹ programming language, where the construct originated, poll/submit has been demonstrated to be useful for implementing a variety of client-server patterns, including a download manager with cooperative cancellation.

The outline of this paper is:

- In Section 2, we give an overview of our main idea.
- In Section 3, we review CP and present its typing rules and operational semantics in a form that is convenient for our later development.
- In Section 4, we introduce poll/submit, a session-centric control structure for many-to-one interaction in CP, together with its syntax, typing rules, and operational semantics. We close this section with metatheoretic results of preservation and progress, which we have mechanized in Lean.
- In Section 5, we demonstrate the expressiveness of poll/submit using three examples that show how to encode constructs from earlier works in our system: coexponentials [6], Qian’s system of classical effects [5], and a strengthened linear reformulation of shared linear state [7]. For each, we derive the corresponding typing rules in our calculus, adapting the source construct where needed to fit our dynamic-pool setting.
- In Section 6, we relate poll/submit to previous approaches to races, fixed points, shared state, and the *(AnonLang)* programming language, where the construct originated.

2 The Main Idea

This paper adds the poll/submit control structure to CP, i.e., the process calculus of session types based on classical linear logic that Wadler introduced in *Propositions as Sessions* [8]. This section explains the construct informally, assuming no familiarity with CP, while the rest of this paper explains it more precisely. We first recall how CP programs run (Section 2.1), then state the problem poll/submit solves (Section 2.2), describe the poll/submit construct in general terms (Section 2.3), show a concrete example (Section 2.4), and end with the construct’s limitations (Section 2.5).

2.1 Background: Programs as Trees of Processes

In CP [8], a running program is a collection of independent, sequential processes. Each process holds local variables, where every variable contains an *endpoint* of a *channel*. Processes interact only through channels. Each channel has exactly two endpoints, held by two different processes, and is a private wire between the two processes. Only channel endpoints, rather than channels themselves, are first-class objects. Thus, whenever we say that a process or variable refers to a channel, we actually mean that it refers to an endpoint of that channel.

Representing processes as nodes and channels as undirected edges results in a connected cycle-free graph, with exactly one path between any two nodes, i.e., an unrooted tree as shown in Figure 1. This is a consequence of how programs in CP are constructed and not an imposed constraint. This tree shape ensures CP programs are deadlock free. However, it also restricts what programs can be expressed, which we revisit in Section 2.5.

¹Anonymized for review

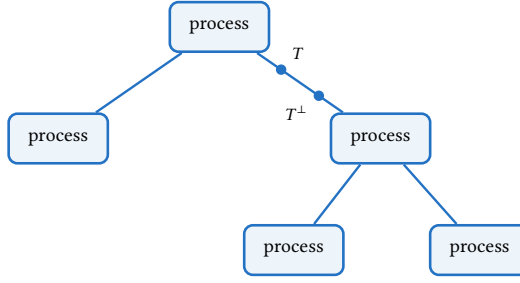


Fig. 1. A program is a tree of processes (nodes) joined by channels (undirected edges). Every channel has two endpoints, held by different processes and operated under dual session types T and $T^\perp$.

Channels allow communication in both directions via the following set of operations. A process can *choose* between alternative ways to continue the session, and the process at the other endpoint *case-selects* to follow whichever way was chosen. This is how a program branches—for example, to report that a stream has ended instead of yielding another value. A process can also *send* or *receive* another channel endpoint, which is what lets channels carry structure, and a process can *close* a channel once its session ends.

For all of this, the two endpoints of a channel must cooperate: when one side sends, the other must receive; when one side offers a choice, the other must choose between the offered options. *Session types* statically enforce this agreement. They prescribe the order, branching, and repetition of the operations on a channel, and they give the two endpoints *dual* types—duality being the operation that swaps sending for receiving, offering for choosing, and so on.

These operations suffice to represent the usual programming types as channels running agreed protocols (e.g., data like numbers and lists, codata like functions and streams, and any combination of the two). Manipulations of these values, such as construction, destructuring, and function application are implemented by communication steps between processes. Thus, in CP, manipulating values and communicating between processes are the *same* activity. A consequence of this unification is that structures such as lists are not blocks of memory that we walk by chasing pointers. Instead, they are communicated one step at a time, revealing themselves gradually. For example, a list's tail may still be undecided while we already hold its head.

2.2 The Problem

The problem we address in this paper is many-to-one interaction of many independent clients communicating with one server, where the server must react to whichever client becomes ready.

As a running example for this section, take a log console. Concurrent client processes emit log messages, which are collected and displayed by a single console server process. The emitters run independently and may terminate themselves or spawn new emitters, so the console knows neither how many of them there will be nor which of them will emit next, but it must handle messages in the order in which they arrive.

CP cannot express such a server because the calculus has no construct whose outcome depends on which interaction becomes ready. Poll/submit is the construct this paper proposes to achieve this.

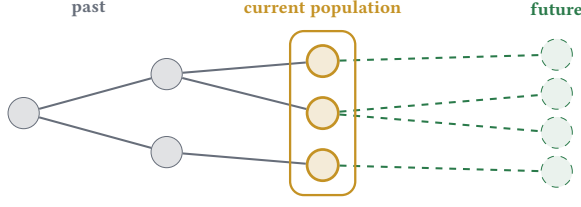


Fig. 2. A recursive client value read as the evolution of a client population.

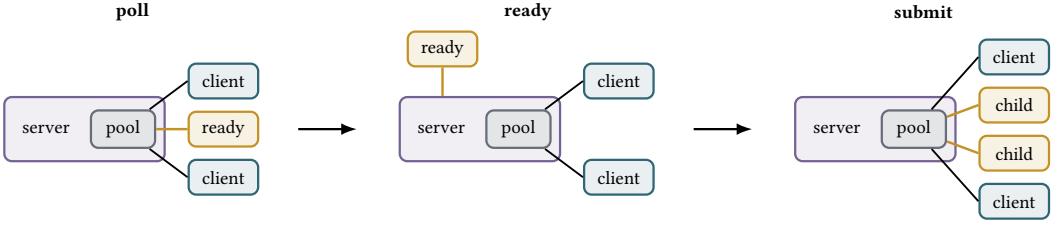


Fig. 3. Nondeterministic recursion as a pool traversal.

2.3 The Idea

The intuition of our proposed approach has three parts: treat a whole population of clients as a value, traverse the population nondeterministically using a pool, and keep the traversal tied to the structure of the client population.

First: Populations of Clients as Values. A population of clients can be assigned a *least-fixed-point type*. These least-fixed-point types are recursive types that may recur an unbounded number of times. We read the tree formed by values of this type, as shown in Figure 2, not as static data but as the *evolution of the population of clients* where parent nodes produce child nodes after communication. Due to linearity, a parent node stops existing once its child nodes are produced. Hence, a node in this tree is a particular client at a particular moment, and the alternative forms it can take are the choices available in its protocol. A node with no children is a client terminating and leaving the population. A node that interacts and produces one child is a client communicating with the server and continuing, or equivalently a client replacing itself with a new client. A node with multiple children is a client that adds new clients to the population. Unfolded over time, a path down the tree is the history of a client's and all its ancestors' interactions with the server.

Second: The Traversal. To traverse the tree that represents the evolution of the population of clients we use a pattern similar to depth-first and breadth-first traversal in that we maintain a set of pending nodes (initially just the root) and repeatedly take a node out, process it, and put its children back in. However, where depth-first traversal keeps these nodes in a stack, and breadth-first traversal keeps them in a queue, our traversal keeps them in a *pool* that hands back whichever node becomes ready. Poll takes a ready node out of the pool, and submit puts its children back in (as shown in Figure 3). The traversal order is determined not by the values themselves but by *when* they communicate. We call this *nondeterministic recursion*.

Third: The Restriction. To make sure that the least fixed-point types truly represent the evolution of the client population, we require that, for each poll followed by a submit issued by the server,

the clients added by the submit must be produced by the client obtained by the poll. The server cannot arbitrarily synthesize new clients to be added to the pool.

2.4 Example: Programming a Log Console

We now apply the idea to our running example introduced in Section 2.2. The population of log emitters is described by the following least fixed-point type, written as a data type in pseudocode:

```
data Client {
  End
  Split(Client, Client)
  Emit(Message, Client)
}
```

A value of type Client is a client of the log console. It may end its participation with End, split into two clients that both continue under the same protocol with Split, or emit one message and continue with Emit.

The console server is implemented by the following pseudocode, which is explained in the next few paragraphs:

```
poll(client) {
  ready(c) ⇒ match c {
    End ⇒ submit()
    Split(left, right) ⇒ submit(left, right)
    Emit(msg, rest) ⇒ { display the message; submit(rest) }
  }
  empty ⇒ { all clients have ended }
}
```

This console server is written with the new poll/submit control structure. The general operation of this construct works as follows. Poll creates a pool, puts a root client into the pool, and waits. When a pooled client becomes ready, the **ready** branch runs: it receives that client, interacts with it, and ends with a submit, which returns zero or more child clients to the pool and resumes waiting. When the pool is empty, the **empty** branch runs instead, and the loop is over.

Thus, for this code, the server's interaction with a client works as follows. The **ready** branch binds a ready client to *c*, matches on it, and does one of the following:

- For End, we submit nothing to the pool, the client terminates, and the pool shrinks.
- For Split, we submit both the left and right children to the pool, and the pool grows.
- For Emit, we display the message *msg* and submit the continuation *rest*, so the same client rejoins the pool.

This traversal exhibits the interleaving we would expect from a log console. Emit nodes from a single client are displayed in order because the continuation of a client enters the pool only after the message has been displayed, but messages from independent clients (which are spawned by Split) interleave arbitrarily, based only on when they Emit.

Note that in this example, the communication flows in only one direction: from emitting clients to the console server. However, in general, once a client becomes ready, communication can go in both directions. For example, later in the paper, we implement mutually exclusive access to shared linear state, where a client takes the contents of a cell, holds it exclusively, and returns a replacement before other clients are served (Section 5.3).

$$\begin{array}{ll}
A, B ::= X \mid X^\perp \mid \mathbf{1} \mid \perp \mid A \otimes B \mid A \wp B \mid A \oplus B \mid A \& B \mid \mathbf{0} \mid \top \mid \mu X. F[X] \mid \nu X. F[X] \\
(X)^\perp = X^\perp & (X^\perp)^\perp = X \\
(\mathbf{1})^\perp = \perp & (\perp)^\perp = \mathbf{1} \\
(A \otimes B)^\perp = A^\perp \wp B^\perp & (A \wp B)^\perp = A^\perp \otimes B^\perp \\
(A \oplus B)^\perp = A^\perp \& B^\perp & (A \& B)^\perp = A^\perp \oplus B^\perp \\
(\mathbf{0})^\perp = \top & (\top)^\perp = \mathbf{0} \\
(\mu X. F[X])^\perp = \nu X. (F[X^\perp])^\perp & (\nu X. F[X])^\perp = \mu X. (F[X^\perp])^\perp
\end{array}$$

Fig. 4. The session-type grammar and linear negation.

2.5 Structural Constraints

The poll/submit construct imposes a structure on the communication that allows us to preserve deadlock freedom and type safety:

- *The pool is homogeneous.* Every client in it must have the same type, since the ready branch handles whichever client emerges without knowing in advance which one it will be. However, this is not a significant constraint. The recursive type that represents clients can contain a sum type that distinguishes between different types of clients.
- *The process graph is a tree.* Two clients of one server cannot communicate directly with each other. A direct channel between two clients of the same server would create a communication cycle. Preventing this ensures the communication graph always forms a tree as in the original CP, which prevents deadlocks.

3 Background

The formal system used in the rest of the paper is an extension of CP (a process calculus based on classical linear logic [8]) that adds poll and submit. This section therefore recalls the fragment of classical linear logic that we use in this paper, namely the multiplicative-additive fragment with least and greatest fixed points added, together with the CP syntax and its operational semantics.

3.1 Session Types

Session types are propositions of classical linear logic that are read as protocols for channels. In our typing judgments, a process holds the channels named on the left-hand side of the sequent. We describe each proposition from the producer's point of view: if a consumer holds a channel of type A , then the process on the other end of that channel, the producer, behaves according to A . The available type formers are shown in Figure 4.

The types are interpreted as follows:

- X and $X^\perp$ are type variables and dual type variables.
- A producer of $\mathbf{1}$ terminates the process behind the channel and closes it. A producer of $\perp$ waits for the consumer to terminate the process behind the channel and close it.
- A producer of $A \otimes B$ sends a channel of type A and then proceeds as B . A producer of $A \wp B$ receives a channel to produce A on, and then proceeds as B . Notably, $A^\perp \wp B$ is also written as $A \multimap B$, a linear function.
- A producer of $A \oplus B$ chooses between A and B and then proceeds as the chosen side. A producer of $A \& B$ offers that choice and then proceeds as the chosen side.
- $\mathbf{0}$ chooses among zero alternatives, so it is the impossible or absurd type, a type with no possible past. Its dual $\top$ offers zero alternatives, so it is a type with no possible future.

$$P, Q ::= x \leftrightarrow y \mid vxy. (P \mid Q) \mid x[] . 0 \mid x().P \mid x[y].(P \mid Q) \mid x(y).P \mid x[inl].P \mid x[inr].P \mid x.case(P, Q) \mid x.case()$$

Fig. 5. The nonrecursive CP terms used in this paper. Code is shown in blue; propositions remain black throughout.

- In $\mu X. F[X]$ (the least fixed point), the producer controls the number of unfoldings and the consumer uses recursion. In $\nu X. F[X]$ (the greatest fixed point), the producer uses recursion and the consumer controls the number of unfoldings.

Following Baelde [1], for $\mu X. F[X]$ and $\nu X. F[X]$, we require the parameter X to occur only positively in F (i.e., $F[X]$ may mention X , but not $X^\perp$). This monotonicity condition ensures that least and greatest fixed points are well behaved.

3.2 CP Syntax

We use Wadler's CP syntax [8], with two small notational choices that have become common in later CP-style calculi: a symmetric cut binder $vxy. (P \mid Q)$ and the direct-send abbreviation $x\langle y \rangle.P$ for $x[z].(y \leftrightarrow z \mid P)$ [2, 3]. In Figure 5, we show the core forms for this system. We introduce the recursive commands poll and submit later in Section 4, so the fixed point types above do not yet appear in the term rules.

The heart of the calculus is the $x[y].(P \mid Q)$ (send) and $z(w).R$ (receive) commands. The send command communicates with a receive command if x and z are endpoints of the same channel. When they rendezvous, a new channel for the transmitted value is created with its endpoints bound to y and w . The receiver continues as R with both z and w in scope. The sender continues as two independent processes P and Q but with only y in scope in P (producing the transmitted value) and only x in scope in Q (continuing the main protocol).² All other variables in scope of the sender get divided between P and Q (i.e., variables are treated linearly). Unlike λ -calculus, CP does not have expressions. Instead, it has commands that communicate over channels but do not themselves evaluate to a value. Thus to produce a value and send it, the sender must spawn a process (P) that will transmit the value over the newly created channel (possibly involving multiple communication steps).

The forwarder $x \leftrightarrow y$ takes two names bound to existing channel endpoints and connects them to each other so the output of one is the input of the other and vice versa. The cut $vxy. (P \mid Q)$ creates a fresh channel, binds its endpoints as x and y , and runs P and Q in parallel with x in scope in P and y in scope in Q and all the other variables in scope divided between P and Q in the same way as in the send command.

The close command $x[] . 0$ closes the channel x and ends the current process, while $x().P$ waits for the peer to do so before continuing as P . The selection commands $x[inl].P$ and $x[inr].P$ commit to one branch of a binary choice, and the branching command $x.case(P, Q)$ reacts to the branch chosen by the peer. Finally, $x.case()$ eliminates a channel of type 0 : if such a channel existed, the current configuration would already be impossible.

3.3 Typing Rules

The traditional presentation of CP writes judgments as $P \vdash x_1 : B_1, \dots, x_n : B_n$, which matches how logicians write sequents (i.e., $\vdash B_1, \dots, B_n$). However, in order to interpret $x_1, \dots, x_n$ as variables in

²Note that the order of x and y is the reverse of P and Q . This convention is from CP and is due to P frequently being much smaller than Q .

$$\begin{array}{c}
\frac{}{x : A, y : A^\perp \vdash x \leftrightarrow y} \text{LINK} \quad \frac{\Gamma, x : A \vdash P \quad \Delta, y : A^\perp \vdash Q}{\Gamma, \Delta \vdash vx y. (P \mid Q)} \text{CUT} \quad \frac{}{x : \perp \vdash x[] . 0} \perp \\
\\
\frac{\Gamma \vdash P}{\Gamma, x : 1 \vdash x().P} 1 \quad \frac{\Gamma, y : A, x : B \vdash P}{\Gamma, x : A \otimes B \vdash x(y).P} \otimes \quad \frac{\Gamma, y : A \vdash P \quad \Delta, x : B \vdash Q}{\Gamma, \Delta, x : A \wp B \vdash x[y].(P \mid Q)} \wp \\
\\
\frac{\Gamma, x : A \vdash P \quad \Gamma, x : B \vdash Q}{\Gamma, x : A \oplus B \vdash x.\text{case}(P, Q)} \oplus \quad \frac{\Gamma, x : A \vdash P}{\Gamma, x : A \& B \vdash x[\text{inl}].P} \&_1 \quad \frac{\Gamma, x : B \vdash P}{\Gamma, x : A \& B \vdash x[\text{inr}].P} \&_2 \\
\\
\frac{}{\Gamma, x : 0 \vdash x.\text{case}() } 0
\end{array}$$

Fig. 6. Typing rules for the nonrecursive CP, written with all channels on the left of the turnstile.

scope instead of named results, we move them to the left of the turnstile and take the dual of each of their types. Thus our type judgments are of the form:

$$x_1 : A_1, \dots, x_n : A_n \vdash P$$

This judgment means that P consumes the channels currently in scope and handles them to completion.

What would be $P \vdash x_1 : B_1, \dots, x_n : B_n$ in the original presentation becomes $x_1 : B_1^\perp, \dots, x_n : B_n^\perp \vdash P$ in our presentation. Intuitively, the right-sided (original) sequents describe how types may be produced, while left-sided (our) sequents describe how types may be consumed. While this means that our typing rules are flipped compared to the traditional CP presentations, this makes many forms more intuitive to interpret as programming constructs. CP does not have expressions (i.e., forms with a natural *result*) and instead consists of commands, which are naturally read as *consuming*.

The typing rules then are as shown in Figure 6. LINK is forwarding: it connects one endpoint to another endpoint of the dual type. CUT spawns two processes around a fresh channel. Together, CUT and $\wp$ are the logical sources of concurrency in CP: both allocate fresh channels and split work across processes. The 1 and $\perp$ rules are the two sides of session termination: one side closes, the other side waits for that close. The multiplicative rules express channel passing. In $\otimes$, the peer sends us a fresh channel, which we bind as y before continuing; in $\wp$, we create a fresh channel ourselves, send one endpoint away, and continue in parallel on both resulting threads. The additive rules express binary choice. The $\oplus$ rule says that if we can continue correctly after either label the peer may choose, then we can branch on that choice. The two $\&$ rules say that if we are free to choose the branch ourselves, it is enough to justify the chosen continuation. Finally, the 0 rule corresponds to absurdity: a channel of type 0 cannot arise in a well-typed run, so no continuation needs to be provided.

Note that variables are treated linearly, so in the CUT and $\wp$ rules, the variables in the environment are linearly divided into Γ and Δ and given to P and Q , respectively.

3.4 Operational Semantics

A runtime state is a multiset of *active processes*. An active process has the form $\langle \gamma \mid P \rangle$, where the runtime channel environment γ is a finite mapping from local variables to channel identifiers. A single binding is written $x \mapsto i$, meaning that the variable x is one of the endpoints of the channel named by the identifier i . We write $\text{fv}(P)$ for the set of free channel variables of P , with the usual

$$\begin{aligned}
& \mathcal{M}, \langle \gamma_1, \gamma_2 \mid vxy. (P \mid Q) \rangle \longrightarrow \mathcal{M}, \langle \gamma_1, x \mapsto i \mid P \rangle, \langle \gamma_2, y \mapsto i \mid Q \rangle \\
& \text{where } \text{dom}(\gamma_1) = \text{fv}(P) \setminus \{x\}, \text{dom}(\gamma_2) = \text{fv}(Q) \setminus \{y\}, \text{ and } i \text{ is fresh} \\
& \mathcal{M}, \langle x \mapsto i, y \mapsto j \mid x \leftrightarrow y \rangle, \langle \gamma, u \mapsto i \mid P \rangle \longrightarrow \mathcal{M}, \langle \gamma, u \mapsto j \mid P \rangle \\
& \mathcal{M}, \langle x \mapsto i, y \mapsto j \mid x \leftrightarrow y \rangle, \langle \gamma, v \mapsto j \mid Q \rangle \longrightarrow \mathcal{M}, \langle \gamma, v \mapsto i \mid Q \rangle \\
& \mathcal{M}, \langle x \mapsto i \mid x[.]0 \rangle, \langle \gamma, y \mapsto i \mid y().P \rangle \longrightarrow \mathcal{M}, \langle \gamma \mid P \rangle \\
& \mathcal{M}, \langle \gamma_1, \gamma_2, x \mapsto i \mid x[y].(P \mid Q) \rangle, \langle \gamma_3, z \mapsto i \mid z(w).R \rangle \longrightarrow \mathcal{M}, \langle \gamma_1, y \mapsto i' \mid P \rangle, \langle \gamma_2, x \mapsto i \mid Q \rangle, \langle \gamma_3, z \mapsto i, w \mapsto i' \mid R \rangle \\
& \text{where } \text{dom}(\gamma_1) = \text{fv}(P) \setminus \{y\}, \text{dom}(\gamma_2) = \text{fv}(Q) \setminus \{x\}, \text{dom}(\gamma_3) = \text{fv}(R) \setminus \{z, w\}, \\
& \text{and } i' \text{ is fresh} \\
& \mathcal{M}, \langle \gamma_1, x \mapsto i \mid x[\text{inl}].P \rangle, \langle \gamma_2, y \mapsto i \mid y.\text{case}(Q, R) \rangle \longrightarrow \mathcal{M}, \langle \gamma_1, x \mapsto i \mid P \rangle, \langle \gamma_2, y \mapsto i \mid Q \rangle \\
& \mathcal{M}, \langle \gamma_1, x \mapsto i \mid x[\text{inr}].P \rangle, \langle \gamma_2, y \mapsto i \mid y.\text{case}(Q, R) \rangle \longrightarrow \mathcal{M}, \langle \gamma_1, x \mapsto i \mid P \rangle, \langle \gamma_2, y \mapsto i \mid R \rangle
\end{aligned}$$

Fig. 7. Operational semantics for the nonrecursive CP fragment.

binding conventions: $vxy. (P \mid Q)$ binds x in P and y in Q , while $x(y).P$ and $x[y].(P \mid Q)$ bind y in P (while x is merely a reference). We require $\text{dom}(\gamma) = \text{fv}(P)$ at all times: the runtime channel environment tells us exactly which variables the process uses. Channel identifiers, i , come from an infinite set, so a reduction may choose a fresh identifier when needed.

The reductions are shown in Figure 7. In each rule, either one or two processes are selected from the multiset of active processes and are replaced with new, reduced processes. $\mathcal{M}$, the rest of the multiset, stays unchanged. Note that there is no separate reduction rule for $x.\text{case}()$ because a channel of type 0 cannot arise in a closed well-typed configuration.

Each reduction mirrors one proof transformation. CUT allocates a fresh channel and exposes its two endpoints to the spawned processes. The two LINK rules remove a forwarder by redirecting one neighboring process from one channel identifier to the other; when both sides have peers, either identifier may be chosen as the survivor. Close/wait removes a finished session. The send/receive rule performs one communication step, preserving the original channel for the continuation and creating a fresh channel for the transmitted endpoint. The two choice rules simply follow the selected label. These reductions preserve the environment/body invariant $\text{dom}(\gamma) = \text{fv}(P)$, and in any closed configuration every channel identifier appears exactly twice: once at each endpoint of the channel it names.

4 Poll and Submit

We now present the central contribution of this paper: the poll/submit control structure introduced informally in Section 2. The server calls **poll** to initialize a pool of clients and wait for a ready client. After interacting with a client, the server calls **submit** to add zero or more of that client's recursive children to the pool and resume waiting. The clients use $x[\text{ready}].P$ to announce that they are ready, then interact with the server. We first explain the syntax and intuition, then illustrate the construct with an example before giving its typing rules and operational semantics.

4.1 Syntax and Intuition

The three new forms are shown in Figure 8, and we explain them in the order in which they are usually executed.

Poll and initial clients. Before the server calls the poll command, it already holds a set (usually of size one) of channel endpoints in variables $x_1, \dots, x_n$ that are connected to processes that will become clients in the initial pool. These server-side variables all have the same least fixed-point

```

442  $P, Q ::= \dots$ 
443       |  $x[\text{ready}].P$ 
444       |  $\text{poll}(x_1, \dots, x_n)[s](P) \{ \text{ready}_p(x, t) \Rightarrow Q, \text{empty}(t) \Rightarrow R \}$ 
445       |  $\text{submit}_p(x_1, \dots, x_n)[s](P)$ 

```

Fig. 8. The forms for connection and nondeterministic recursion.

type of the form $\mu X. F[X]$, while the corresponding variables on the client-side all have the dual of that type, which is the greatest fixed-point type $\nu X. (F[X^\perp])^\perp$. Thus each server-client pair already forms the two sides of one recursive protocol. The pool is initialized with pre-existing clients; it does not create the clients or their channels. (Indeed, as we will see later, clients added to the pool by submit are always spawned by the client processes themselves and are not spontaneously created by the pool or the server.)

Using these $x_1, \dots, x_n$, the server invokes poll with

```

456
457        $\text{poll}(x_1, \dots, x_n)[s](P) \{$ 
458          $\text{ready}_p(x, t) \Rightarrow Q,$ 
459          $\text{empty}(t) \Rightarrow R$ 
460        $\}$ 

```

This consumes (in the linear logic sense) the variables $x_1, \dots, x_n$ and transfers their endpoints into a newly created *pool*, which then waits for readiness signals from the client processes holding the dual endpoints.

The pool itself is part of the server's process state, and contains the endpoints of those clients that have not been served yet. It is not a channel, a process, or a first-class value, and the server has no variable whose value is the pool. Instead, the submit command specifies the pool to submit to by the label p that is introduced in the $\text{ready}_p(x, t) \Rightarrow Q$ branch of the poll command.

As the server recurs through the client tree, it threads state between rounds of polling using s , P , and t , which we explain later.

The client's readiness signal. Each endpoint in the pool belongs to a channel whose other endpoint is in a client and has the dual greatest fixed-point type. The only operation a client can do on an endpoint of such type is to call the $[\text{ready}]$ command:

```

475        $u[\text{ready}].P'$ 

```

This signals to the server that the client is ready to be served. Once the server selects the client and goes into its **ready** branch, the client continues as P' , where u remains in scope of P' . The type of u changes to the unfolded version of the greatest fixed-point type, and the communication with the server proceeds on u according to that type. (Recall that variables change type as they are interacted with in CP.)

The ready branch. While polling, the pool watches for clients to become ready. While no clients are ready, the server blocks. Once one or more clients are ready, the server selects one of those ready clients and runs the poll command's ready_p branch:

```

485        $\text{ready}_p(x, t) \Rightarrow Q$ 

```

The endpoint of the selected client is removed from the pool and bound as x in Q . The other endpoints remain in the pool. On the client side, the selected client resolves its $[\text{ready}]$ command and continues as P' .

The channel connecting the server to the selected client is now an ordinary channel bound to x in Q and u in P' on the server and client, respectively. The recursive protocol has been unfolded once on both the client and server, and they communicate according to those unfolded types (e.g., using the ordinary CP commands from Section 3). Any endpoints of type X from the original $\mu X. F[X]$ that the server obtains during this interaction are the children of the client in the tree of clients, as described in Section 2.

Submit and the next polling round. The **ready_p** branch of the server concludes by submitting the children of the client $(y_1, \dots, y_n)$ to the pool (identified by p) by calling:

$$\text{submit}_p(y_1, \dots, y_n)[s](P)$$

Submit transfers control back to the corresponding **poll** and begins the next polling round, while also consuming the variables $y_1, \dots, y_n$ and adding their endpoints to the pool. (Here again, s and P are for threading state through the recursion, which we explain later.)

If the list of submitted clients is empty, the selected client has no children, and no new clients are added to the pool, but the server continues polling clients that were left in the pool by the previous poll.

The empty branch. Once the pool contains no endpoints (not just that no clients are ready), the server enters the **empty** branch of poll:

$$\text{empty}(t) \Rightarrow R$$

The server closes the pool and continues as R , with the final state bound as t . (If the initial $x_1, \dots, x_n$ list to poll was empty, the **ready** branch is never entered, and the **empty** branch runs immediately.)

Explicit server state. In the poll form, the $[s](P)$ and t are used to thread the server's state. (This is inspired by the handling of recursion state in Baelde [1].) Because variables are linear and there may be many iterations of poll/submit, we do not allow the **ready** and **empty** branches to implicitly capture channels from outside the poll, since doing so would significantly complicate the typing and reduction rules and make cut elimination difficult. Instead, the process P inherits the entire environment at **poll** (except for $x_1, \dots, x_n$) and uses it to construct the initial state on a fresh channel whose endpoint s is in-scope in P . Whichever branch (**ready** or **empty**) runs receives the dual endpoint to s as t . At a submit, the same notation constructs the state for the next iteration. This mechanism may be used to plainly pass the values of the environment into the recursion as a tuple, or to construct some initial state to be incremented on each polling round.

4.2 Introductory Example

A small instructive example is a nondeterministic traversal of binary trees whose leaves carry channels of type A . Let

$$T_A = \mu X. A \oplus (X \otimes X)$$

Seen as a data structure, a value of type T_A is either a leaf of type A or a node with two children of the original type. Seen as a client protocol, a client of type T_A either chooses to perform a single A -interaction (after which it terminates), or splits into two independent clients.

To keep the example readable, we treat as generic the initialization of the state, the handling of leaves, and the disposal of the final state by abbreviating them as **init** $\langle s \rangle$, **handle** $\langle n, t \rangle$, and **dest** $\langle t \rangle$, respectively. Their interfaces are as follows:

$$\frac{}{\Gamma, s : S^\perp \vdash \text{init}\langle s \rangle} \quad \frac{t : S \vdash P}{n : A, t : S \vdash \text{handle}\langle n, t \rangle.P} \quad \frac{t : \perp \vdash P}{t : S \vdash \text{dest}\langle t \rangle.P}$$

For example, if we are appending the leaves to a flat list, then S would be a type of a channel on which we can produce the list, $\text{init}\langle s \rangle$ would transmit on s the channel to which the list is to be produced (captured from the environment), $\text{handle}\langle n, t \rangle$ would send on t the value n as the next element of the list, and $\text{dest}\langle t \rangle$ would close t by signaling the end of the list.

Then, using poll/submit, the server traverses the tree using the following:

```

poll⟨n0⟩[s](init⟨s⟩) {
  readyp(n, t) ⇒ n.case(
    handle⟨n, t⟩. submitp⟨⟩[s](s ↔ t),
    n(n1). submitp⟨n1, n⟩[s](s ↔ t),
    empty(t) ⇒ dest⟨t⟩. t[.0]
  )
}

```

The code starts with a singleton pool containing the initial tree n_0 . The initial state is created by $\text{init}\langle s \rangle$ on s and then threaded through the loop. Whenever some pending subtree becomes ready, the **ready** branch receives an unfolded version of it as n with t being the current recursion state (either constructed by $\text{init}\langle s \rangle$ or threaded by **submit** from the previous iteration). In the first/left branch of the **case**, n becomes a leaf of type A , so the abbreviation **handle** processes that leaf and **submit** is called with no new clients. In the second/right branch, n becomes an internal node of type $X \otimes X$. Receiving on n exposes two recursive subtrees with n_1 as the left child and n as the right child (recall that in CP variables change types upon interaction). Then, **submit** is called with both n_1 and n submitted to the pool. When no subtrees remain, the **empty** branch runs, and **dest** disposes of the state.

This code does not prescribe the order in which leaves are processed. The order is determined only by which client performs **ready** first on a subtree.

4.3 Typing Rules

Before stating the typing rules for poll/submit, we explain how poll exposes a least fixed point while still tracking which recursive children come from the selected client. Consider again the tree protocol from the introductory example in Section 4.2. The channel endpoint that the client waits on (initially the other side of the endpoint n_0) has type:

$$T_A^\perp = \nu X. A^\perp \& (X \wp X)$$

After the client performs **ready**, that endpoint unfolds to:

$$A^\perp \& (T_A^\perp \wp T_A^\perp)$$

So on the server, when the server selects that client at poll, the n in the **ready**_p(n, t) branch gets a corresponding unfolding of the least-fixed-point type $T_A = \mu X. A \oplus (X \otimes X)$. A naive typing rule would unfold this to:

$$A \oplus (T_A \otimes T_A)$$

But this would cause a problem for submit. In this case, the child clients have the same least-fixed-point type that we initially passed to poll: T_A . However, if submit accepts any client of type T_A , then the server could synthesize a client of type T_A that was not part of the client tree and submit it to the pool, which would prevent the least fixed-point types from truly representing the evolution of the client population.

To resolve this, we use a simple mechanism that prevents the server from synthesizing clients on its own. Instead of unfolding to T_A , poll introduces a fresh type variable Y and exposes the selected client as

$$A \oplus (Y \otimes Y)$$

$$\begin{array}{c}
\frac{}{\emptyset; x : A, y : A^\perp \vdash x \leftrightarrow y} \text{LINK} \quad \frac{\Pi_1; \Gamma, x : A \vdash P \quad \Pi_2; \Delta, y : A^\perp \vdash Q}{\Pi_1, \Pi_2; \Gamma, \Delta \vdash vxy.(P \mid Q)} \text{CUT} \quad \frac{}{\emptyset; x : \perp \vdash x[] . 0} \perp \\
\\
\frac{\Pi; \Gamma \vdash P}{\Pi; \Gamma, x : 1 \vdash x().P} 1 \quad \frac{\Pi; \Gamma, y : A, x : B \vdash P}{\Pi; \Gamma, x : A \otimes B \vdash x(y).P} \otimes \quad \frac{\Pi_1; \Gamma, y : A \vdash P \quad \Pi_2; \Delta, x : B \vdash Q}{\Pi_1, \Pi_2; \Gamma, \Delta, x : A \wp B \vdash x[y].(P \mid Q)} \wp \\
\\
\frac{\Pi; \Gamma, x : A \vdash P \quad \Pi; \Gamma, x : B \vdash Q}{\Pi; \Gamma, x : A \oplus B \vdash x.\text{case}(P, Q)} \oplus \quad \frac{\Pi; \Gamma, x : A \vdash P}{\Pi; \Gamma, x : A \& B \vdash x[\text{inl}].P} \&_1 \quad \frac{\Pi; \Gamma, x : B \vdash P}{\Pi; \Gamma, x : A \& B \vdash x[\text{inr}].P} \&_2 \\
\\
\frac{}{\Pi; \Gamma, x : 0 \vdash x.\text{case}()} 0
\end{array}$$

Fig. 9. The nonrecursive CP typing rules with pool environments.

$$\begin{array}{c}
\frac{\Pi; \Gamma, x : G[vX.G[X]] \vdash P}{\Pi; \Gamma, x : vX.G[X] \vdash x[\text{ready}].P} \text{READY} \\
\\
\frac{\Pi_1; \Gamma, s : S^\perp \vdash P \quad p : (Y, S); x : F[Y], t : S \vdash Q \quad \Pi_2; t : S \vdash R \quad Y \text{ fresh} \quad n \geq 0}{\Pi_1, \Pi_2; \Gamma, x_1 : \mu X.F[X], \dots, x_n : \mu X.F[X] \vdash \text{poll}\langle x_1, \dots, x_n \rangle[s](P)\{\text{ready}_p(x, t) \Rightarrow Q, \text{empty}(t) \Rightarrow R\}} \text{POLL} \\
\\
\frac{\Pi; \Gamma, s : S^\perp \vdash P \quad n \geq 0}{p : (Y, S), \Pi; \Gamma, x_1 : Y, \dots, x_n : Y \vdash \text{submit}_p\langle x_1, \dots, x_n \rangle[s](P)} \text{SUBMIT}
\end{array}$$

Fig. 10. Typing rules for connection and nondeterministic recursion.

Because Y is fresh, the recursive children exposed by this polling round are the only channels of type Y .

Pool labels are used to track which fresh marker Y was introduced by a poll, so that the corresponding submit can only submit clients of that type. After a poll, the ready_p branch remembers which fresh marker Y was introduced and the suspended information needed to resume polling when the matching submit happens. To support this, our typing judgment carries not only the standard type environment Γ but also a *pool environment* Π that tracks the pool labels p . Thus the typing judgment has the form:

$$\Pi; \Gamma \vdash P$$

The new pool environment Π contains bindings of the form $p : (X, S)$, where p is the pool label and (X, S) is a *pool type*. In this pool type, X is the fresh type variable introduced in the unfolding in the ready_p branch, and S is the type of the loop state that is threaded through the polling rounds.

Figure 9 shows the nonrecursive CP rules modified with this extra environment, while Figure 10 shows the rules for poll and submit. In all cases, p is treated linearly.

In Figure 9, the LINK and $\perp$ rules allow no poll points, unary rules thread Π through unchanged, multiplicative rules split and merge Π linearly, and the additive case rule keeps the same pool environment on both premises because only one branch will ever run. Only the 0 rule may discard poll points for the same reason it discards channels: a process with a variable of that type can never actually run.

In Figure 10, the **READY** rule is the only way to use an endpoint connected to the server, which is of a greatest fixed-point type. At the type level it unfolds the greatest fixed point once. At the operational level it is the signal that this client is ready to be selected by poll.

In the **POLL** rule, the current environment Γ is in scope in only the auxiliary process P , which produces the initial recursion state. The rule then starts a fresh nondeterministic recursion. When a client becomes ready and is selected, the **ready** branch receives three resources: a variable x holding the selected client, a variable t holding the current recursion state, and a fresh pool label p . The variable x has type $F[Y]$ instead of $\mu X. F[X]$, where the fresh variable Y marks the recursive children that may later be passed to submit.

The side condition requires that Y be fresh. This ensures that Y is treated opaquely in the **ready_p** branch and is used only by submit.

Note that the pool environment is split between P and R (in the form of Π_1 and Π_2), but not Q . This means that a poll can be nested inside another poll, but the inner poll cannot use the outer pool label. Thus the inner poll must exhaust its own pool before returning to the outer poll, and serving the inner pool's clients cannot be interleaved with serving the outer pool's clients. The reason for this constraint is the need to preserve the linearity of pool labels.

The **SUBMIT** rule is how the server resumes polling from within the **ready_p** branch. It consumes the pool label p introduced by the **ready_p** branch, adds zero or more child clients $x_1, \dots, x_n$ of type Y to the pool, and uses P to produce the next state on a fresh channel s . Even a **ready_p** branch that returns no clients must still call submit, because the pool may contain other clients that have not yet been handled. Also, because pool labels are linear, the **ready_p** branch can neither forget to submit, nor submit twice.

4.4 Operational Semantics

To add poll/submit to the operational semantics, we extend runtime environments from Figure 7 with pool labels. An active process within the multiset of active processes $\mathcal{M}$ now has the form $\langle \pi; \gamma \mid P \rangle$, where γ —the *runtime channel environment*—is the ordinary variable-to-channel map from Section 3, and π —the *runtime pool environment*—is a finite mapping from pool labels to *pool cells*. We write $\text{fv}(P)$ for the free *channel* variables of P and $\text{fpv}(P)$ for the free *pool labels*. A well-formed process requires $\text{dom}(\gamma) = \text{fv}(P)$ and $\text{dom}(\pi) = \text{fpv}(P)$.

A *pool cell* records the suspended information of a poll while running its **ready** branch:

$$(\pi; i_1, \dots, i_n; x; t_Q; Q; t_R; R)$$

It records the runtime pool environment π reserved for the **empty** branch, the channel identifiers $i_1, \dots, i_n$ for the clients still waiting in the pool, and a copy of both the **ready** branch as variable names x, t_Q and a process code Q and the **empty** branch as a variable name t_R and a process code R .

Figure 11 shows the nonrecursive CP reductions modified with this extra component π , while Figure 12 shows the rules for **ready**, **poll**, and **submit**. As in typing, the multiplicative rules split pool environments linearly, while the additive rules preserve the same pool environment across the surviving branch.

The only rules that actually inspect the pool environment are the three new rules in Figure 12. One for the **ready** branch of poll, one for the **empty** branch of poll, and one for **submit**.

On the client side, the $x[\text{ready}].P$ operation reduces only with a matching **poll** on the server side. On the server side, there are three cases to consider.

The first rule is for the **ready** branch of poll. If a pooled client meets a waiting **poll** by performing **[ready]** on the same channel i_j (bound to u in the client and y_j in the server), that client is detached from the pool and handed to the **ready** branch as an ordinary local channel. The remainder of the pool is frozen in a pool cell stored in p together with the **empty** branch's saved pool environment.

$$\begin{aligned}
& \mathcal{M}, \langle \pi_1, \pi_2; \gamma_1, \gamma_2 \mid vxy.(P \mid Q) \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma_1, x \mapsto i \mid P \rangle, \langle \pi_2; \gamma_2, y \mapsto i \mid Q \rangle \\
& \quad \text{where } \text{dom}(\pi_1) = \text{fpv}(P), \text{dom}(\pi_2) = \text{fpv}(Q), \text{dom}(\gamma_1) = \text{fv}(P) \setminus \{x\}, \\
& \quad \text{dom}(\gamma_2) = \text{fv}(Q) \setminus \{y\}, \text{ and } i \text{ is fresh} \\
& \mathcal{M}, \langle \emptyset; x \mapsto i, y \mapsto j \mid x \leftrightarrow y \rangle, \langle \pi; \gamma, u \mapsto i \mid P \rangle \longrightarrow \mathcal{M}, \langle \pi; \gamma, u \mapsto j \mid P \rangle \\
& \mathcal{M}, \langle \emptyset; x \mapsto i, y \mapsto j \mid x \leftrightarrow y \rangle, \langle \pi; \gamma, v \mapsto j \mid Q \rangle \longrightarrow \mathcal{M}, \langle \pi; \gamma, v \mapsto i \mid Q \rangle \\
& \mathcal{M}, \langle \emptyset; x \mapsto i \mid x[.].0 \rangle, \langle \pi_1; \gamma_1, y \mapsto i \mid y().P \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma_1 \mid P \rangle \\
& \mathcal{M}, \langle \pi_1, \pi_2; \gamma_1, \gamma_2, x \mapsto i \mid x[y].(P \mid Q) \rangle, \langle \pi_3; \gamma_3, z \mapsto i \mid z(w).R \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma_1, y \mapsto i' \mid P \rangle, \langle \pi_2; \gamma_2, x \mapsto i \mid Q \rangle, \langle \pi_3; \gamma_3, z \mapsto i, w \mapsto i' \mid R \rangle \\
& \quad \text{where } \text{dom}(\pi_1) = \text{fpv}(P), \text{dom}(\pi_2) = \text{fpv}(Q), \text{dom}(\pi_3) = \text{fpv}(R), \\
& \quad \text{dom}(\gamma_1) = \text{fv}(P) \setminus \{y\}, \text{dom}(\gamma_2) = \text{fv}(Q) \setminus \{x\}, \\
& \quad \text{dom}(\gamma_3) = \text{fv}(R) \setminus \{z, w\}, \text{ and } i' \text{ is fresh} \\
& \mathcal{M}, \langle \pi_1; \gamma_1, x \mapsto i \mid x[\text{inl}].P \rangle, \langle \pi_2; \gamma_2, y \mapsto i \mid y.\text{case}(Q, R) \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma_1, x \mapsto i \mid P \rangle, \langle \pi_2; \gamma_2, y \mapsto i \mid Q \rangle \\
& \mathcal{M}, \langle \pi_1; \gamma_1, x \mapsto i \mid x[\text{inr}].P \rangle, \langle \pi_2; \gamma_2, y \mapsto i \mid y.\text{case}(Q, R) \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma_1, x \mapsto i \mid P \rangle, \langle \pi_2; \gamma_2, y \mapsto i \mid R \rangle
\end{aligned}$$

Fig. 11. The nonrecursive CP reductions with pool environments.

$$\begin{aligned}
& \mathcal{M}, \langle \pi_3; \gamma_1, u \mapsto i_j \mid u[\text{ready}].P' \rangle, \langle \pi_1, \pi_2; \gamma_2, y_1 \mapsto i_1, \dots, y_n \mapsto i_n \mid \text{poll}\langle y_1, \dots, y_n \rangle[s](P)\{\text{ready}_P(x, t) \Rightarrow Q, \text{empty}(t) \Rightarrow R\} \rangle \\
& \quad \longrightarrow \\
& \mathcal{M}, \langle \pi_3; \gamma_1, u \mapsto i_j \mid P' \rangle, \langle \pi_1; \gamma_2, s \mapsto i' \mid P \rangle, \langle p \mapsto (\pi_2; i_1, \dots, i_{j-1}, i_{j+1}, \dots, i_n; x; t_Q; Q; t_R; R); x \mapsto i_j, t \mapsto i' \mid Q \rangle \\
& \quad \text{where } \text{dom}(\pi_1) = \text{fpv}(P), \text{dom}(\pi_2) = \text{fpv}(R), 1 \leq j \leq n, \text{ and } i' \text{ is fresh} \\
\hline
& \mathcal{M}, \langle \pi_1, \pi_2; \gamma \mid \text{poll}\langle \rangle[s](P)\{\text{ready}_P(x, t) \Rightarrow Q, \text{empty}(t) \Rightarrow R\} \rangle \longrightarrow \mathcal{M}, \langle \pi_1; \gamma, s \mapsto i \mid P \rangle, \langle \pi_2; t \mapsto i \mid R \rangle \\
& \quad \text{where } \text{dom}(\pi_1) = \text{fpv}(P), \text{dom}(\pi_2) = \text{fpv}(R), \text{ and } i \text{ is fresh} \\
\hline
& \mathcal{M}, \langle \pi_1, p \mapsto (\pi_2; i_1, \dots, i_m; x; t_Q; Q; t_R; R); \gamma, y_1 \mapsto j_1, \dots, y_n \mapsto j_n \mid \text{submit}_P\langle y_1, \dots, y_n \rangle[s](P) \rangle \\
& \quad \longrightarrow \\
& \mathcal{M}, \langle \pi_1, \pi_2; \gamma, z_1 \mapsto i_1, \dots, z_m \mapsto i_m, y_1 \mapsto j_1, \dots, y_n \mapsto j_n \mid \text{poll}\langle z_1, \dots, z_m, y_1, \dots, y_n \rangle[s](P)\{\text{ready}_P(x, t) \Rightarrow Q, \text{empty}(t) \Rightarrow R\} \rangle \\
& \quad \text{where } z_1, \dots, z_m \text{ are fresh variable names}
\end{aligned}$$

Fig. 12. Operational semantics for **ready**, **poll**, and **submit**.

This reduction rule alone is the source of nondeterminism in **poll**/**submit**. Any channel i_j that has a ready client can be selected by the server.

The second rule is for the **empty** branch of **poll**. If the pool is empty, the **empty** branch runs. The auxiliary process P produces the final state on the fresh channel i bound to s , and the **empty** branch receives the other endpoint of i as t together with the pool environment π_2 reserved for it when the initial **poll** was called.

The third rule is for **submit**. The remembered client channels $i_1, \dots, i_m$ in the pool cell p are given fresh variables $z_1, \dots, z_m$. Then execution continues with a reconstructed **poll** on a set of clients that merges the remembered clients $z_1, \dots, z_m$ with the freshly submitted clients $y_1, \dots, y_n$.

4.5 Metatheoretic Results

We prove the metatheoretic properties of preservation and progress for our system, thereby proving that the system is type-safe and deadlock-free. The system itself, the definitions, and the proofs of these metatheoretic results are all mechanized in Lean and are available in the supplementary material.

4.5.1 Definitions. We call a runtime configuration *rooted* when it has a single reference to a distinguished external channel ω . For the metatheory in this section, we consider only programs where the interface of this external channel is termination, i.e., the type of the internal endpoint assigned to ω is $\perp$ (which would be seen from the outside as its dual 1). More complicated external

interfaces are possible that support things such as I/O, but that is beyond the scope of this work, as that would require formalizing the behavior of the external world in order to prove the properties shown in this section.

A rooted configuration $\mathcal{M}$ is *well formed* when all active processes in it are properly scoped with regard to channel variables and pool labels, and all used channels (other than ω) are referenced exactly twice (once by each endpoint). A well-formed configuration is *well typed* when every active process $\langle \pi; \gamma \mid P \rangle$ has a typing derivation $\Pi; \Gamma \vdash P$ where π and γ match Π and Γ . A runtime configuration $\mathcal{M}$ is *reachable* if it can be reached after some number of reduction steps starting from an initial configuration $\langle \emptyset; x \mapsto \omega \mid P \rangle$ that is well typed as $\emptyset; x : \perp \vdash P$.

The *process-channel graph* of a rooted configuration treats (1) active processes as vertices and (2) (non- ω) channels as edges, where the edge for a channel is between the vertices for the two processes that have the channel's endpoints in their runtime environments (whether in the channel environment or the pool environment).

4.5.2 Theorems. First, we show that types are preserved by reduction, and all reachable configurations are well typed, which implies type safety.

Theorem 4.1 (Preservation). *If $\mathcal{M}$ is well typed and $\mathcal{M} \longrightarrow \mathcal{N}$, then $\mathcal{N}$ is well typed.*

PROOF. By case analysis of each of the reduction rules. □

Corollary 4.2 (Reachable Configurations are Well Typed). *If $\mathcal{M}$ is reachable, then it is well typed.*

Next, before proving progress, we show that process-channel graphs of reachable runtime configurations are always trees. This allows us to induct over the structure of that tree in the progress proof.

Lemma 4.3 (Tree Shape). *The process-channel graph of every reachable configuration is a tree.*

PROOF. By induction over the sequence of reductions and case analysis of each of the reduction rules. The initial configuration has one process and zero (non- ω) channels. Every reduction that adds a process also adds a channel and every reduction that removes a process also removes a channel. Thus the number of processes is the number of (non- ω) channels plus one, and all reductions preserve connectedness. Hence, by graph theory, the process graph of a reachable configuration is a tree. □

This now allows us to prove progress, which implies deadlock freedom.

Theorem 4.4 (Progress). *If $\mathcal{M}$ is reachable, then either $\mathcal{M} \longrightarrow \mathcal{N}$ for some $\mathcal{N}$, or it is a terminal configuration*

$$\mathcal{M} = \langle \emptyset; z \mapsto \omega \mid z[\] . 0 \rangle$$

for some variable name z .

PROOF. The variable name that is referenced by the first command of a process is called its *subject*. Most commands, such as $x[y].(P|Q)$ or $x(y).P$ have a single subject (here x), **poll** and **submit** may have multiple.

First, we *root* the process-channel graph of $\mathcal{M}$ at the process that contains ω and inspect the root process. If the first command of the root process is a cut, a link, an empty **poll**, or any **submit**, we found a reduction. Otherwise, if its subject is the variable bound to ω , then it is a terminal configuration. Otherwise, we take the process on the other side of any of its subjects and note an *invariant* for the following induction: For this child process, its parent's subject is bound to a channel connected to this process.

The induction step for non-root processes in the process-channel graph is as follows. If the first command of the process is a cut, a link, an empty **poll**, or any **submit**, we found a reduction. Otherwise, we first check the subject(s) of the current process. If any of the subjects is bound to the channel to the parent, then by the invariant, we found a reduction because both the parent and the child are attempting to perform an operation with each other on the same channel. If all subjects are not bound to the channel to the parent, then they are bound to channels connecting the current process to a process that is a child of the current process in the process-channel graph. The invariant is then true for these child processes and thus we find a reduction by the induction hypothesis in any of them. $\square$

5 Prior Work as Examples

We now demonstrate the expressiveness of poll/submit with three examples. Each rebuilds, inside our calculus, a central construct from an earlier paper, appropriately adapted to fit our setting. The three constructs are coexponentials [6], Qian's classical effects [5], and shared linear state [7]—three approaches to nondeterminism in linear logic, each programmed here as a macro layer over poll/submit. Taken in order, each of these systems can express things that the previous ones cannot. This section serves a dual purpose of both giving examples of how to use poll/submit and showing how to recover the characteristic behavior of these other systems.

We build all three examples the same way: we turn their modal types into fixed-point types, and their operations into macros over our calculus. The typing derivations of all of the macros against their respective rules have been mechanized in Lean. However, given the technical differences between the source papers and our formalism, we do not prove operational equivalence.

As a brief overview, the key ideas of each of these systems are as follows:

- **Coexponentials** [6]. The coexponential $\wr A$ is the type of a pool of clients sharing a single server, each served exactly once. A client connects to the server, is handled to completion, and then leaves the pool, so the only nondeterminism is the order in which clients connect.
- **Classical effects** [5]. Classical effects remove the restriction that a client interacts only once. A waiting client instead chooses among a fixed set of named effects, each of which sends a request and receives a response before the client returns to the pool, so a client may interact repeatedly. This suffices for repeated interaction, but since each interaction is a single request answered by a single response, it cannot implement a mutex.
- **Shared linear state** [7]. Shared linear state takes a different approach that dispenses with the central server. Clients share a mutex cell directly: a client locks the cell, takes its contents, holds them exclusively, and returns a replacement before any other client is served. This is the interaction order a mutex requires and that classical effects cannot express, but the clients coordinate through shared state rather than two-sided communication channels.

5.1 Coexponentials

The first example comes from *Client-Server Sessions in Linear Logic* [6], which introduces *coexponentials*, written $\wr A$ and $\wr A$. Here $\wr A$ is the type of a pool of A -clients, namely a channel held by the server, while each waiting client locally holds the dual channel to the server $\wr A$.

The clients in the coexponential pool may *connect* and turn into A -sessions to interact with the server. The main limitation of this system is that once a client turns into an A -session, it may never rejoin the pool. The nondeterminism only applies to the order in which the A -sessions are started. After an A -session is started, it is deterministically handled by the server to completion and the client's session terminates.

The rules for forming such a pool are the duals of weakening, dereliction, and contraction for ordinary exponentials in linear logic:

$$\frac{}{\vdash \dot{\iota}A} \quad \frac{}{\vdash \Gamma, \dot{\iota}A} \quad \frac{\vdash \Gamma, A \quad \vdash \Gamma, \dot{\iota}A \quad \vdash \Delta, \dot{\iota}A}{\vdash \Gamma, \Delta, \dot{\iota}A}$$

These three $\dot{\iota}$ -rules say that a pool may be empty, may consist of exactly one client, or may merge two subpools into one.

The server itself is formed by the strong co-promotion rule, so called in [6] because it comes from their fixed-point account of coexponentials:

$$\frac{\vdash \Gamma, B \quad \vdash B^\perp, \perp \quad \vdash B^\perp, A, B}{\vdash \Gamma, \dot{\iota}A}$$

This rule threads an internal state B through a one-client-at-a-time server loop. Each of the premises, respectively, says that a local environment Γ initializes a state, a separate process can finalize it, and a third process can use one current state to handle one A -client while producing the next state.

To translate their type to our system, we represent coexponentials by the following fixed-point types:

$$\dot{\iota}A = \mu X. 1 \oplus (A \oplus (X \otimes X)) \quad \dot{\iota}A = \nu X. \perp \& (A \& (X \wp X))$$

Figures 13 and 14 show how we translate these rules. The duals of weakening, dereliction, and contraction become **CLOSE**, **CONNECT**, and **SPLIT**, respectively, while strong co-promotion becomes **SERVE**. (Note that when moving propositions from the right to the left side of the sequents, $\dot{\iota}$ swaps with its dual $\dot{\iota}$.) Where the original right-sided formulation reads with a constructive intuition, our equivalent left-sided formulation reads observationally. What was “the pool may be empty” becomes “a client may disconnect” (**CLOSE**). What was “turn one A -client into a pool” becomes “a client may connect as A ” (**CONNECT**). What was “merge two pools” becomes “a client may split into two clients” (**SPLIT**).

Figure 14 gives the macro expansions for **close**, **connect**, **split**, and **serve** in our calculus, such that they satisfy the typing rules in Figure 13. The macros **close**, **connect**, and **split** implement the client operations on the pre-existing server channel x . Each of them starts by signaling readiness with $x[\text{ready}]$ and then proceeding to select the respective branch of the protocol via a sequence of $x[\text{inl}]$ and $x[\text{inr}]$ commands. Finally, **close** concludes by terminating the client, **connect** by implementing an A -session by the process P , and **split** by binding a new channel to the server as y and spawning two parallel client processes: P and Q . Note that the macro parameters have different roles: x is a reference position, y is a binding position, and P and Q are process code.

The expansion of **serve** in Figure 14 is the most substantial as it implements the coexponential server using poll/submit. The process parameters I , D , and H specify how to construct and destruct the threaded state, and how to handle a single A -client while updating the state, respectively. The heart of the implementation is in the **ready** branch which analyzes three cases of a client x . In the first case, the server acknowledges the client’s termination, and resumes polling. In the second case, the process H handles the client’s A -session and updates the threaded state. In the third case, $x(x_1)$ receives a new client channel and adds both clients to the pool, resuming polling.

An important side note is that the original paper studies two variants. The three client rules at the start of this subsection are the tree-shaped ones, and they are also the direct duals of the ordinary exponential rules. Later, in order to describe a sequential server loop, the paper replaces the last two client rules by the list-shaped rule:

$$\frac{\vdash \Gamma, A \quad \vdash \Delta, \dot{\iota}A}{\vdash \Gamma, \Delta, \dot{\iota}A}$$

$$\begin{array}{c}
\frac{}{x : iA \vdash \text{close}\langle x \rangle} \text{CLOSE} \quad \frac{\Gamma, x : A \vdash P}{\Gamma, x : iA \vdash \text{connect}\langle x, P \rangle} \text{CONNECT} \quad \frac{\Gamma, y : iA \vdash P \quad \Delta, x : iA \vdash Q}{\Gamma, \Delta, x : iA \vdash \text{split}\langle x, y, P, Q \rangle} \text{SPLIT} \\
\\
\frac{\Gamma, s : S^\perp \vdash I \quad t : S \vdash D \quad s : S^\perp, t : S, x : A \vdash H}{\Gamma, x : iA \vdash \text{serve}\langle x, s, t, I, D, H \rangle} \text{SERVE}
\end{array}$$

Fig. 13. The same coexponential rules, now annotated with the macros that realize them. In `split`, the fresh channel `y` is bound in the first continuation.

```

close⟨x⟩      =  x[ready]. x[inl]. x[.].0
connect⟨x, P⟩ =  x[ready]. x[inr]. x[inl]. P
split⟨x, y, P, Q⟩ = x[ready]. x[inr]. x[inr]. x[y].(P | Q)

serve⟨x, s, t, I, D, H⟩ =
  poll⟨x⟩[s](I){
    readyP(x, t) ⇒ x.case(
      x(). submitP⟨⟩[s](s ↔ t),
      x.case(
        submitP⟨⟩[s](H),
        x(x1). submitP⟨x, x1⟩[s](s ↔ t)
      ),
    empty(t) ⇒ D
  }

```

Fig. 14. Macros implementing the coexponential operations. Macro names and macro parameters are shown in red; primitive syntax remains blue.

For our example we keep the one-client-at-a-time server reading but retain the tree-shaped client rules. The list-shaped client rules would enforce a sequencing where each unfolding reveals just one client of type A together with the remainder of the pool, which forces clients to be spawned in order with no way to query which client is ready. The original paper gets around this sequencing by adding a shuffling of this list in the runtime semantics. However, this shuffling does not reflect client readiness. By keeping the tree shape, clients independently emerge in sibling branches of the tree, which enables poll/submit to handle them in the order they become ready.

5.2 Classical Effects

The second example comes from Qian's dissertation [5], whose classical effects remove the main limitation of coexponentials: a client need not permanently leave the pool after one interaction. A client chooses an effect from a defined set, sends a request on the channel to the server, receives a response, and then the channel returns to its initial protocol. Thus a single client may interact with the server repeatedly, with other clients served between its interactions.

Qian's system is parameterized by an effect environment Θ and by a final continuation type C . However, C is a fixed-length tuple of the final results from all clients, which, as previously mentioned, is not compatible with a dynamically sized pool of clients. Thus, we do not implement C in our adaptation and instead focus on the effect environment Θ .

The environment Θ maps effect names to request-response pairs. We write $A_i : B_i$ for the interface of effect i : a client sends an A_i -request and receives a B_i -response. Dually, the server receives an A_i -request and sends back a B_i -response. We present an instance of the protocol with

$$\begin{array}{c}
\frac{}{x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash \text{close}\langle x \rangle} \text{CLOSE} \quad \frac{\Gamma, y : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash P \quad \Delta, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash Q}{\Gamma, \Delta, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash \text{split}\langle x, y, P, Q \rangle} \text{SPLIT} \\
\\
\frac{\Gamma, y : A_1^\perp \vdash P \quad \Delta, z : B_1, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash Q}{\Gamma, \Delta, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash \text{effect}_1\langle x, y, z, P, Q \rangle} \text{EFFECT}_1 \\
\\
\frac{\Gamma, y : A_2^\perp \vdash P \quad \Delta, z : B_2, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash Q}{\Gamma, \Delta, x : \text{Srv}_{A_1:B_1, A_2:B_2} \vdash \text{effect}_2\langle x, y, z, P, Q \rangle} \text{EFFECT}_2 \\
\\
\frac{\Gamma, s : S^\perp \vdash I \quad t : S \vdash D \quad t : S, a_1 : A_1, h_1 : B_1^\perp \wp S^\perp \vdash H_1 \quad t : S, a_2 : A_2, h_2 : B_2^\perp \wp S^\perp \vdash H_2}{\Gamma, x : \text{Cli}_{A_1:B_1, A_2:B_2} \vdash \text{serve}\langle x, s, t, I, D, H_1, H_2 \rangle} \text{SERVE}
\end{array}$$

Fig. 15. A two-effect adaptation of classical effects, already annotated with the macros that realize it. In effect_i , the response channel z is bound in the second continuation; in serve , the request names a_1, a_2 and the handler channels h_1, h_2 are bound in H_1, H_2 .

only two effects to illustrate the idea. At the end of this subsection, we show how to generalize to an arbitrary number of effects as supported by Qian [5].

The fixed-point types for a server and clients with only two effects then are:

$$\text{Srv}_{A_1:B_1, A_2:B_2} = \nu X. \perp \& ((X \wp X) \& ((A_1^\perp \wp (B_1 \otimes X)) \& (A_2^\perp \wp (B_2 \otimes X))))$$

$$\text{Cli}_{A_1:B_1, A_2:B_2} = \mu X. \mathbf{1} \oplus ((X \otimes X) \oplus ((A_1 \otimes (B_1^\perp \wp X)) \oplus (A_2 \otimes (B_2^\perp \wp X))))$$

The greatest fixed point $\text{Srv}_{A_1:B_1, A_2:B_2}$ is the client-side view of the server, while its dual least fixed point $\text{Cli}_{A_1:B_1, A_2:B_2}$ is the server-side view of the clients that are polled. The important difference from coexponentials is that each effect branch returns to X , so the selected client is submitted back to the pool after its request has been answered.

Looking at $\text{Srv}_{A_1:B_1, A_2:B_2}$ from a client's perspective, the $\perp$ branch closes the client, the $X \wp X$ branch splits it into two independent clients, and the i -th effect branch is a linear function from an A_i -request to a B_i -response together with a renewed channel to the server X . Dually, $\text{Cli}_{A_1:B_1, A_2:B_2}$ is the server-side type of a polled client: $\mathbf{1}$ is client termination, $X \otimes X$ is client splitting, and $A_i \otimes (B_i^\perp \wp X)$ packages a request with a function that accepts the response and returns the next version of the client.

Figure 15 gives the adapted left-sided rules annotated with the macros that realize them. The first four rules are the client-side operations, corresponding to BoW, BoC, and BoA from the original paper. **CLOSE** is BoW with the final continuation removed, and **SPLIT** keeps the sharing behavior of BoC but returns two homogeneous clients. The two **EFFECT_i** rules are instances of BoA for two effects. The last rule is for the server side and corresponds to DI from the original paper: it initializes a state, destroys the final state when the pool becomes empty, and supplies a handler body for each effect.

Figure 16 gives the macro expansions. Each client-side macro first signals readiness with $x[\text{ready}]$, then chooses the appropriate branch of the recursive protocol. The **close** macro terminates the client, **split** creates two residual clients, and **effect_i** sends a request, waits for a response, and continues with the same server-channel type. The **serve** macro is a poll/submit recursion over the client pool. Client terminations and splitting are handled as we did for coexponentials. In an effect branch, the handler receives the current state and the request, and produces both the response and the next state. The response is sent back to the client, and the residual client is submitted back to the pool.

```

981      close⟨x⟩ = x[ready]. x[inl]. x[].0
982      split⟨x, y, P, Q⟩ = x[ready]. x[inr]. x[inl]. x[y].(P | Q)
983      effect1⟨x, y, z, P, Q⟩ = x[ready]. x[inr]. x[inr]. x[inl]. x[y].(P | x(z). Q)
984      effect2⟨x, y, z, P, Q⟩ = x[ready]. x[inr]. x[inr]. x[inr]. x[y].(P | x(z). Q)
985
986      serve⟨x, s, t, I, D, H1, H2⟩ =
987      poll⟨x⟩[s](I){
988        readyp(x, t) ⇒ x.case(
989          x(). submitp⟨⟩[s](s ↔ t),
990          x.case(
991            x(x1). submitp⟨x, x1⟩[s](s ↔ t),
992            x.case(
993              x(a1). v h1 k1. (H1 | k1(r1). x⟨r1⟩. submitp⟨x⟩[s](s ↔ k1)),
994              x(a2). v h2 k2. (H2 | k2(r2). x⟨r2⟩. submitp⟨x⟩[s](s ↔ k2))
995            ))
996          empty(t) ⇒ D
997        }

```

Fig. 16. Macros implementing the two-effect adaptation of classical effects. Macro names and macro parameters are shown in red; primitive syntax remains blue.

The key parts of the `serve` macro are the effect-handling branches. For example, the first effect branch is:

$$x(a_1). v h_1 k_1. (H_1 \mid k_1(r_1). x\langle r_1 \rangle. \text{submit}_p\langle x \rangle[s](s \leftrightarrow k_1))$$

Here $x(a_1)$ receives the request sent by the client. The cut $v h_1 k_1$ starts one instance of the effect handler, with H_1 running on the handler endpoint h_1 . The parallel process running on the dual endpoint k_1 then receives the handler's response with $k_1(r_1)$, after which the handler produces the next state on k_1 . Finally, $x\langle r_1 \rangle$ sends the response to the client on x , and $\text{submit}_p\langle x \rangle[s](s \leftrightarrow k_1)$ puts the client back into the pool while threading that next state into the next iteration.

To generalize this construction from 2 effects to n effects, let $E_i[X] = A_i \otimes (B_i^\perp \wp X)$. The client type becomes:

$$\mu X. 1 \oplus (X \otimes X) \oplus E_1[X] \oplus \dots \oplus E_n[X]$$

and the server type is its dual. To invoke an effect, a client sends a sequence of `inrs` to select the appropriate effect followed by a `inl`:

$$x[\text{inr}]. \dots x[\text{inr}]. x[\text{inl}]$$

The last effect is reached by `inrs` alone. On the server side, `serve` nests a `case` for each effect.

5.3 Shared Linear State

The third example comes from Rocha and Caires [7], who take a different approach altogether. Instead of a server that offers named effects to a pool of clients, they introduce shareable mutex cells. A client that holds an unlocked mutex (which they call “having full usage”) may terminate and drop the mutex's reference (which they call “releasing” the mutex), split into two independent clients that both reference the mutex (which they call “sharing”), or lock the mutex. When a mutex is locked, its current contents will be removed from the mutex cell and given to the client. In the client that emptied the mutex, the type of the mutex changes so the only possible operation on the

$$\begin{array}{c}
\frac{}{\text{Mutex}_A \vdash} \text{RELEASE} \qquad \frac{\Gamma, \text{Mutex}_A \vdash \quad \Delta, \text{Mutex}_A \vdash}{\Gamma, \Delta, \text{Mutex}_A \vdash} \text{SHARE} \qquad \frac{\Gamma, A, A^\perp \wp \text{Mutex}_A \vdash}{\Gamma, \text{Mutex}_A \vdash} \text{TAKE} \\
\\
\frac{\Gamma, A^\perp \vdash \quad \Delta, \text{Mutex}_A \vdash}{\Gamma, \Delta, A^\perp \wp \text{Mutex}_A \vdash} \text{PUT} \qquad \frac{\Gamma, A^\perp \vdash \quad A \vdash}{\Gamma, \text{Pool}_A \vdash} \text{MUTEX}
\end{array}$$

Fig. 17. A strengthened left-sided presentation of shared linear state. The first four rules describe the client interface; the last rule constructs the mutex handler. Variable names and process code are omitted.

mutex is to put new contents back into it, which unlocks it. Other clients that try to lock the mutex are blocked until the mutex is filled by the client that emptied it.

We can model this mutex as a server that holds the current contents of the mutex, supplies them to clients upon request, and waits for the client to return new contents to be stored in the mutex. However, there is a problem with doing this in classical effects [5]. This is because in classical effects, after a single request from the client to the server *followed* by a single response from the server to the client, the client returns to the pool and the server is available to other clients. However, a mutex requires that the *server* transmits the mutex contents *first*, and only then the client transmits the new mutex contents back to the server. The order of communication required to implement a mutex is opposite to what classical effects force.

The original system from Rocha and Caires [7] stores only positive affine data in the mutex. Since affine types are not available in our purely linear setting, we strengthen the mutex interface by returning the final contents of the mutex to the creator of the mutex instead of discarding it when the last usage of the mutex is released. This eliminates both the positive and affine restrictions. The restricted affine version can be recovered simply by having the original creator dispose of the final contents.

Figure 17 gives the rules for this system, where Mutex_A is the type of the channel endpoint seen by the client, while Pool_A is the type of the channel endpoint seen by the server. Rocha and Caires [7] represent unlocked/filled mutexes with the type $U^\bullet A$ and locked/empty mutexes with the type $U^\circ A$. The former corresponds to our Mutex_A , while the latter corresponds to our $A^\perp \wp \text{Mutex}_A$, which in linear logic is a function from A to a Mutex_A . This directly represents the only operation the client can do on a locked mutex: unlocking the mutex by filling it with an A .

RELEASE and SHARE are the same early disconnect (CLOSE) and client splitting (SPLIT) seen in the previous examples. The new behaviors are TAKE and PUT. TAKE locks the mutex and removes its current contents, while changing the mutex channel into a type with only one operation: unlocking the mutex by setting its new contents. PUT performs that unlocking, while changing the mutex channel into an unlocked mutex type. The server-side MUTEX consumes a client pool Pool_A by serving the clients as they lock and unlock the mutex. In its premises, it requires a way to construct the initial contents of the mutex, and a way to consume the final contents.

To represent such mutexes in our system, we encode Mutex_A and Pool_A as the following dual types:

$$\begin{aligned}
\text{Mutex}_A &= \nu X. \perp \& ((X \wp X) \& (A \otimes (A^\perp \wp X))) \\
\text{Pool}_A &= \mu X. 1 \oplus ((X \otimes X) \oplus (A^\perp \wp (A \otimes X)))
\end{aligned}$$

Figures 18 and 19 then give the annotated rules and macro expansions for the operations on these types. Their behavior is the same as just described for our strengthened variant of Rocha and Caires [7].

$$\begin{array}{c}
\frac{}{x : \text{Mutex}_A \vdash \text{release}\langle x \rangle} \text{RELEASE} \qquad \frac{\Gamma, y : \text{Mutex}_A \vdash P \quad \Delta, x : \text{Mutex}_A \vdash Q}{\Gamma, \Delta, x : \text{Mutex}_A \vdash \text{share}\langle x, y, P, Q \rangle} \text{SHARE} \\
\frac{\Gamma, y : A, x : A^\perp \wp \text{Mutex}_A \vdash P}{\Gamma, x : \text{Mutex}_A \vdash \text{take}\langle x, y, P \rangle} \text{TAKE} \qquad \frac{\Gamma, y : A^\perp \vdash P \quad \Delta, x : \text{Mutex}_A \vdash Q}{\Gamma, \Delta, x : A^\perp \wp \text{Mutex}_A \vdash \text{put}\langle x, y, P, Q \rangle} \text{PUT} \\
\frac{\Gamma, s : A^\perp \vdash I \quad t : A \vdash D}{\Gamma, x : \text{Pool}_A \vdash \text{mutex}\langle x, s, t, I, D \rangle} \text{MUTEX}
\end{array}$$

Fig. 18. The strengthened shared-state rules, now annotated with the macros that realize them. In **TAKE**, the same channel continues at type $A^\perp \wp \text{Mutex}_A$; in **PUT**, the returned state and the recovered mutex are handled in parallel.

```

release⟨x⟩      = x[ready]. x[inl]. x[].0
share⟨x, y, P, Q⟩ = x[ready]. x[inr]. x[inl]. x[y].(P | Q)
take⟨x, y, P⟩    = x[ready]. x[inr]. x[inr]. x(y). P
put⟨x, y, P, Q⟩  = x[y].(P | Q)

mutex⟨x, s, t, I, D⟩ =
  poll⟨x⟩[s](I){
    readyp(x, t) ⇒ x.case(
      x(). submitp⟨⟩[s](s ↔ t),
      x.case(
        x(x1). submitp⟨x, x1⟩[s](s ↔ t),
        x⟨t⟩. x(t′). submitp⟨x⟩[s](s ↔ t′)
      ),
    ),
  empty(t) ⇒ D
}

```

Fig. 19. Macros implementing the strengthened shared-state interface. Macro names and macro parameters are shown in red; primitive syntax remains blue.

In these macro expansions, the server acts as the mutex that clients lock and unlock. A client attempts to lock the mutex by signaling that it is ready to be selected by the server, and whichever client is selected by the server is the one that successfully locks the mutex. All other ready clients block until they are themselves selected. The server handles the lock/unlock sequence using the following branch in the **mutex** macro:

$$x\langle t \rangle. x(t'). \text{submit}_p\langle x \rangle[s](s \leftrightarrow t')$$

It first hands the current contents t of the mutex to the selected client x . The server then waits for the client to return the new contents t' before resubmitting the client to the pool with the new contents, at which point the server is free to provide the contents of the mutex to other clients (i.e., the mutex that the server represents is unlocked again).

5.3.1 Sharing Locked Mutexes. Readers familiar with Rocha and Caires [7] may notice that one operation from that paper is omitted in our adaptation. Their system supports sharing locked mutexes, not just unlocked ones. When sharing a locked mutex, one client obtains an unlocked

reference to the mutex and the other client keeps the locked one. Hence, the invariant that at most one client holds a locked mutex is preserved.

The typing rule for our adaptation of this operation is as follows:

$$\frac{\Gamma, y : \text{Mutex}_A \vdash P \quad \Delta, x : A^\perp \wp \text{Mutex}_A \vdash Q}{\Gamma, \Delta, x : A^\perp \wp \text{Mutex}_A \vdash \text{sharelock}\langle x, y, P, Q \rangle} \text{SHARELOCK}$$

The reason we do not include this rule in Figure 18 is that it can be straightforwardly implemented in the client without requiring any additional server-side code; therefore we do not consider it primitive. The macro expansion that implements this is as follows:

$$\text{sharelock}\langle x, y, P, Q \rangle = \nu x x_1. (Q \mid x_1(v). x\langle v \rangle. \text{share}\langle x, y, P, x \leftrightarrow x_1 \rangle)$$

The cut rebinds the name x for Q , so Q keeps operating its locked mutex under the same name, now mediated through the fresh channel between x and x_1 . The original locked mutex x remains in scope in only the right-hand process (i.e., to the right of “ $\mid$ ”). Once Q unlocks its x by putting back new contents, the right-hand process receives it as v on x_1 and sends it onward to the original mutex x , which unlocks the mutex. Next, share shares the mutex between endpoints x and y and hands y to P while handing x back to Q by linking x to x_1 .

6 Related Work

Propositions as Sessions. The starting point for this paper is Wadler’s *Propositions as Sessions* [8], which presents CP as a process calculus interpretation of classical linear logic. We directly extend CP with the poll/submit control structure to support many-to-one nondeterministic communication based on readiness.

Least and Greatest Fixed Points in Linear Logic. Our fixed-point types follow Baelde [1]. We keep the monotonicity condition and duality between least and greatest fixed points. Where the original paper studies deterministic recursion over least fixed-point types, our central contribution is the introduction of nondeterministic recursion over the same types, with its application to client-server sessions.

A small insight worth noting is the relationship between deterministic and nondeterministic recursion. If we deconstruct a value of a least fixed-point type until we are left holding its N recursive children, *deterministic recursion* will make N recursive calls, each over 1 child, whereas *nondeterministic recursion* will make 1 recursive call (submit) over all N children.

Coexponentials and classical effects. The coexponentials of Qian et al. [6] add modalities for a shared server and a pool of clients, and they support nondeterminism in the order in which client sessions are handled by the server. Their main limitation is that a client is served only once. After finishing its session, a client leaves the population. Thus the Emit branch from the example in Section 2.4, where a client emits a message and rejoins the pool, is something coexponentials cannot support.

Qian’s later system of classical effects [5] adds support for repeated interactions by the same client. A client can repeatedly choose a named effect, send a request, receive a response, and return to the pool. But each effect still has the fixed shape of one request followed by one response. That is insufficient for protocols such as mutexes.

Our system, poll/submit, can express communication patterns similar to coexponentials and classical effects (as demonstrated in Sections 5.1 and 5.2) including repeated interactions by the same client, but our system is also sufficient to implement mutexes, which neither of them can do.

An additional distinction between poll/submit and these systems lies in the structural operations: client termination and splitting. Instead of being primitive, these operations must be explicitly

specified in our client-server fixed-point types and handled explicitly by the poll/submit server. This makes the entrance and exit of clients to and from the pool visible to the server, while in coexponentials and classical effects they are not. Thus a server may, for example, track live clients.

Shared linear state. Rocha and Caires [7] take a different route: instead of giving a server a pool of clients, they add shared state that is guarded by a mutex that is referenced by otherwise independent processes. This captures a key interaction pattern that classical effects cannot express. It is, however, not centered on two-sided session-based communication. Locking and unlocking are operations on a shared cell, with no dual actions performed on opposite endpoints. In contrast, poll/submit keeps the interaction between the server and the selected client as an ordinary session. In Section 5.3, we show that poll/submit can express this mutex pattern. Additionally, our adaptation of their system strengthens it to a fully linear mutex by returning the final cell contents to the creator once the last client has released the mutex. Thus, while their mutex can store only affine types, ours can store both linear and affine types because stored values are handled linearly.

We conjecture that poll/submit is encodable in our strengthened version of shared linear state by storing the state of the server in the mutex and replacing the single server by a wrapper process per client. Each wrapper waits for its original client to be ready before competing for the mutex, running the server's `ready` branch, and finally returning the updated server state to the mutex. This is a non-trivial whole-program translation of the server code that requires machinery outside the core system of this paper, including deterministic recursion and unfolding of least fixed points.

⟨AnonLang⟩ (anonymized for review). Poll/submit originated in *⟨AnonLang⟩*, an experimental programming language based on classical linear logic and session-typed concurrency. It uses the same basic idea as this paper, but with additional practical conveniences, and its source repository contains examples of poll/submit including a download manager with cooperative cancellation. Our work is an effort to formalize the nondeterministic recursion of *⟨AnonLang⟩*.

Towards Races in Linear Logic. Kokke et al. [4] give an account of races in a hypersequent variant of CP called HCPND inspired by bounded linear logic. Such types describe a pool of N clients and a server with N nondeterministically ordered interactions. In their system, N is statically fixed, while in poll/submit the number of clients is determined dynamically.

7 Conclusion

This paper introduces poll/submit, a control structure for readiness-based client-server communication in CP, using nondeterministic recursion over least and greatest fixed points. Least fixed points describe populations of clients, greatest fixed points describe client-side views of the server, and poll/submit is the control structure for nondeterministic recursion.

Our examples show that this mechanism can express the client-server patterns of coexponentials [6], classical effects [5], and shared linear state [7] inside a single session-centric calculus. In each case, the protocol remains an ordinary session type. Structural behavior such as client termination and splitting is made visible to the server instead of being primitive.

Though all three of these systems [5–7] are believed to be terminating, only shared linear state [7] proves it. Likewise, we conjecture that poll/submit is terminating. Two arguments support this. First, our treatment of greatest fixed-point types is proof-theoretically identical to a propositional version of Baelde [1], hence values of least fixed-point types should be finite, unless poll/submit itself diverges. Second, if submit always replaces a client from its corresponding poll with a finite number of structurally smaller clients (which should be enforced by the use of a fresh Y variable), then the multiset of clients in the pool is decreasing with each polling round according to the Dershowitz-Manna ordering. A proof of this is left to future work.

References

- [1] David Baelde. 2012. Least and Greatest Fixed Points in Linear Logic. *ACM Transactions on Computational Logic* 13, 1 (2012), 2:1–2:44. doi:10.1145/2071368.2071370
- [2] Simon Fowler, Wen Kokke, Ornela Dardha, Sam Lindley, and J. Garrett Morris. 2021. Separating Sessions Smoothly. In *32nd International Conference on Concurrency Theory (CONCUR 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 203)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 36:1–36:18. doi:10.4230/LIPIcs.CONCUR.2021.36
- [3] Wen Kokke, Fabrizio Montesi, and Marco Peressotti. 2019. Better Late Than Never: A Fully Abstract Semantics for Classical Processes. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 24:1–24:29. doi:10.1145/3290337
- [4] Wen Kokke, J. Garrett Morris, and Philip Wadler. 2020. Towards Races in Linear Logic. *Logical Methods in Computer Science* 16, 4 (2020), 15:1–15:16. doi:10.23638/LMCS-16(4:15)2020
- [5] Zesen Qian. 2023. *Concurrency and Races in Classical Linear Logic*. Ph.D. Dissertation. Aarhus University.
- [6] Zesen Qian, G. A. Kavvos, and Lars Birkedal. 2021. Client-Server Sessions in Linear Logic. *Proceedings of the ACM on Programming Languages* 5, ICFP, Article 62 (2021), 31 pages. doi:10.1145/3473567
- [7] Pedro Rocha and Luís Caires. 2023. Safe Session-Based Concurrency with Shared Linear State. In *Programming Languages and Systems – ESOP 2023 (LNCS, Vol. 13990)*. Springer, Cham, Switzerland, 421–450. doi:10.1007/978-3-031-30044-8_16
- [8] Philip Wadler. 2014. Propositions as Sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. doi:10.1017/S095679681400001X